

PRODUCT DATA STUDIO PRO

Architecture Overview • Обзор архитектуры

Public-safe architecture for Gumroad and GitHub •
Безопасное публичное описание

Windows 10/11 • Local processing • English / Русский

Product Data Studio Pro 2.2.1

Contents • Содержание

English Architecture Overview

System layers and data flow

Validation and batch workflows

Windows distribution

Repository separation and privacy

Архитектура на русском

Слои системы и поток данных

Проверка и пакетная обработка

Сборка и распространение Windows-версии

Разделение репозитория и
конфиденциальность

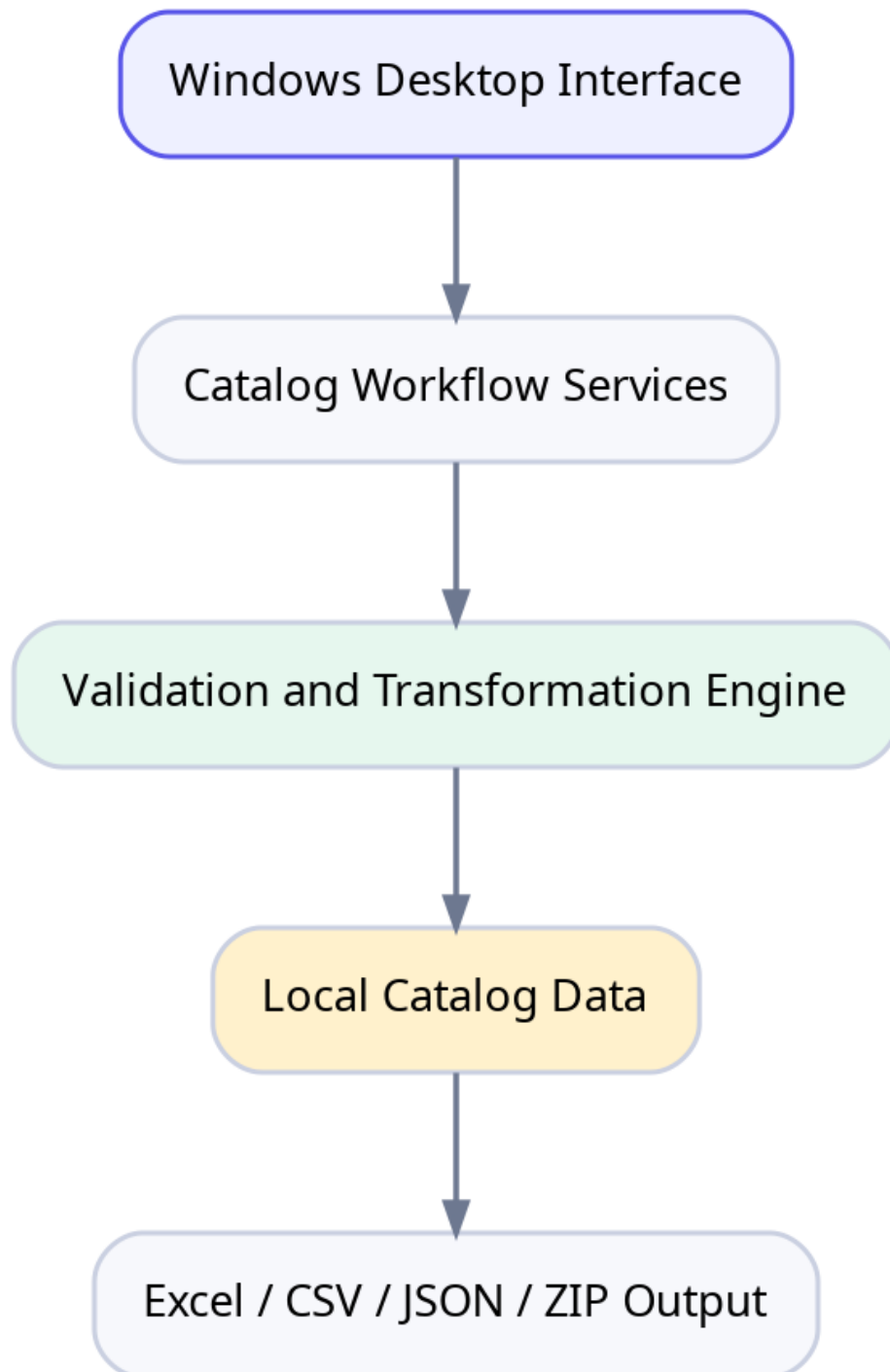
ENGLISH ARCHITECTURE

High-level, public-safe technical overview

Product Data Studio Pro 2.2.1

1. Architecture goals

- Keep catalog-processing logic reusable and testable outside the user interface.
- Process customer files locally and never overwrite source data automatically.
- Support both single-catalog and multi-catalog workflows.
- Separate validation, transformation, comparison, merging and export concerns.
- Package the commercial product as a self-contained Windows application.



High-level architecture from desktop interface to local outputs.

2. Main layers

Layer	Responsibility
Presentation layer	PySide6 windows, dialogs, tables, previews and English/Russian interface.
Application services	Coordinates imports, audits, edits, history, previews and export actions.
Processing core	Reusable cleaning, validation, comparison, merging, batch and export logic.
Working state	pandas DataFrame plus source metadata, mapping, issues, statuses and Undo/Redo history.
Output layer	Excel, CSV, JSON and ZIP reports written to user-selected locations.

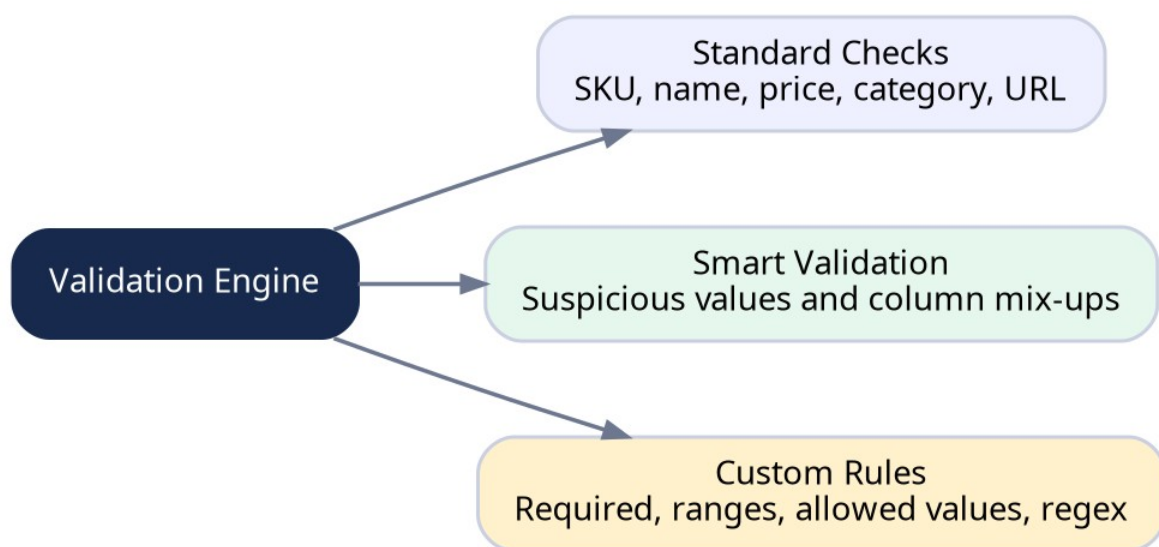
3. Import and catalog state



The operational data flow used by the desktop application.

The active catalog is held as a working DataFrame. Related state includes source path, sheet name, original row number, column mapping, active rules, issues, product status, catalog health and edit history. Source files are treated as read-only inputs.

4. Validation architecture



Three validation layers operate on the same working catalog state.

Standard validation handles predictable structural problems. Smart validation detects suspicious semantic patterns. Custom rules let business users define required fields, ranges, allowed values and regular expressions without changing source code.

5. Editing and history

- 1 A manual edit, bulk operation, safe fix or recipe is requested.
- 2 The application creates an Undo snapshot.
- 3 The working DataFrame is updated.
- 4 Validation is re-run on the changed catalog.
- 5 Issues, product status and health score are refreshed.

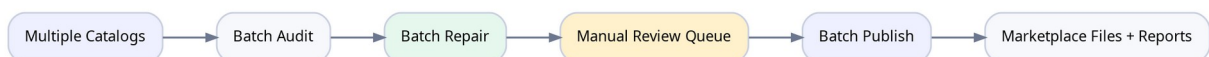
Design principle

User actions are reversible. Large future catalogs may use delta-based history instead of complete DataFrame snapshots.

6. Catalog operations

Service	Architectural role
Compare	Matches products by SKU or another selected key and records field-level changes.
Merge Update	Updates matched products, adds new rows and optionally protects existing values from blank updates.
Split	Partitions one catalog by a group value or fixed row count.
Combine	Unifies schemas from several files and reports duplicate keys.
Transformation Recipes	Applies reusable ordered operations and records every changed cell.

7. Batch architecture



Batch services operate as a staged workflow and preserve per-file results.

Each catalog retains its own DataFrame, mapping, audit result and output metadata. One failed file does not stop the remaining batch. Repair and manual review preserve a detailed change history before publication.

8. Marketplace publishing

- Publishing begins from an audited working catalog.
- A scope selector decides whether Ready, Review or all products are included.
- Platform-specific exporters map the catalog to Shopify or WooCommerce columns.
- Excluded products are written to an optional review report.
- Marketplace outputs are draft or unpublished by default.

9. Local-first privacy

Privacy boundary

Input catalog → local Windows processing → user-selected output folder. The public architecture does not require external catalog upload or telemetry.

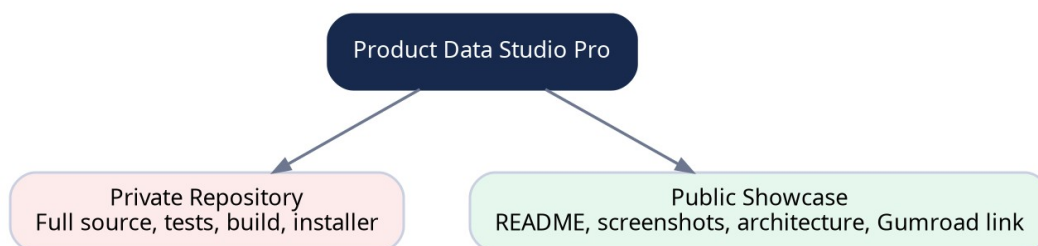
10. Windows distribution



Commercial build and delivery pipeline.

PyInstaller packages the Python application and runtime into a portable Windows folder. Inno Setup creates the installer. The installer, documentation and examples are placed in one customer ZIP for Gumroad.

11. Repository separation



Private implementation and public showcase are intentionally separated.

Repository	Contents
Private repository	Full source code, tests, build scripts, installer configuration and internal release files.
Public showcase	README, screenshots, public architecture, privacy information, system requirements and Gumroad link.

12. Extension points

- Additional marketplace exporters.

- Reusable marketplace mapping profiles.
- Plugin-based validators.
- Product image verification.
- API integrations that use customer-provided credentials.
- AI-assisted text generation as an optional isolated service.

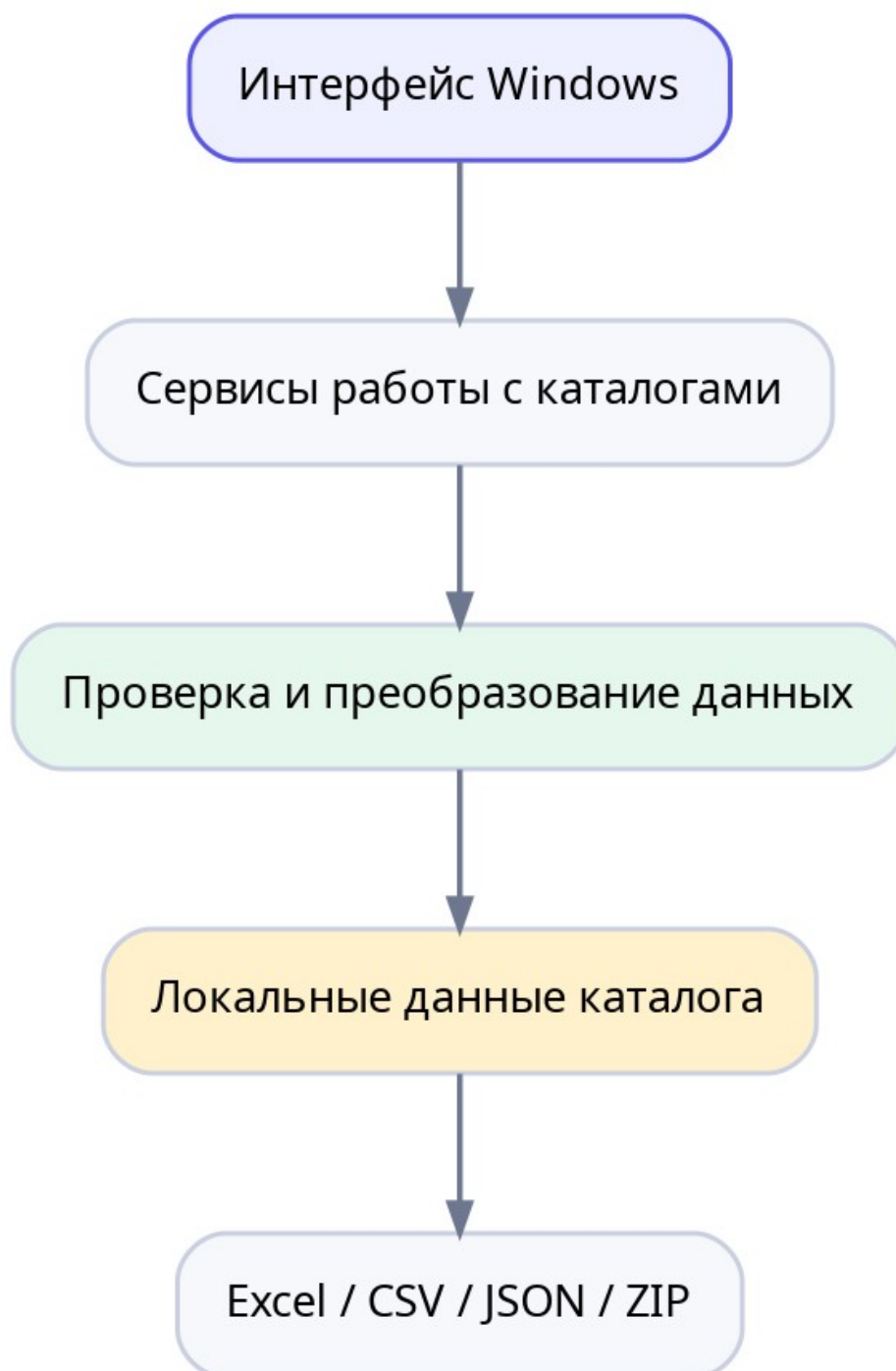
АРХИТЕКТУРА НА РУССКОМ

Высокоуровневое публичное техническое описание

Product Data Studio Pro 2.2.1

1. Цели архитектуры

- Отделить обработку каталогов от пользовательского интерфейса и упростить тестирование.
- Обращаться к файлам локально и не перезаписывать исходные данные автоматически.
- Поддерживать работу как с одним каталогом, так и с пакетами файлов.
- Разделить проверку, преобразование, сравнение, объединение и экспорт.
- Поставлять программу как самостоятельное Windows-приложение.



Высокоуровневая архитектура: от интерфейса до локальных выходных файлов.

2. Основные слои

Слой	Ответственность
Интерфейс	Окна PySide6, диалоги, таблицы, предпросмотр и английский/русский язык.
Сервисы приложения	Координация импорта, проверки, изменений, истории и экспорта.
Ядро обработки	Очистка, проверка, сравнение, объединение, пакетные процессы и экспорт.
Рабочее состояние	pandas DataFrame, источник, сопоставление, ошибки, статусы и история Undo/Redo.
Выходной слой	Excel, CSV, JSON и ZIP в выбранных пользователем папках.

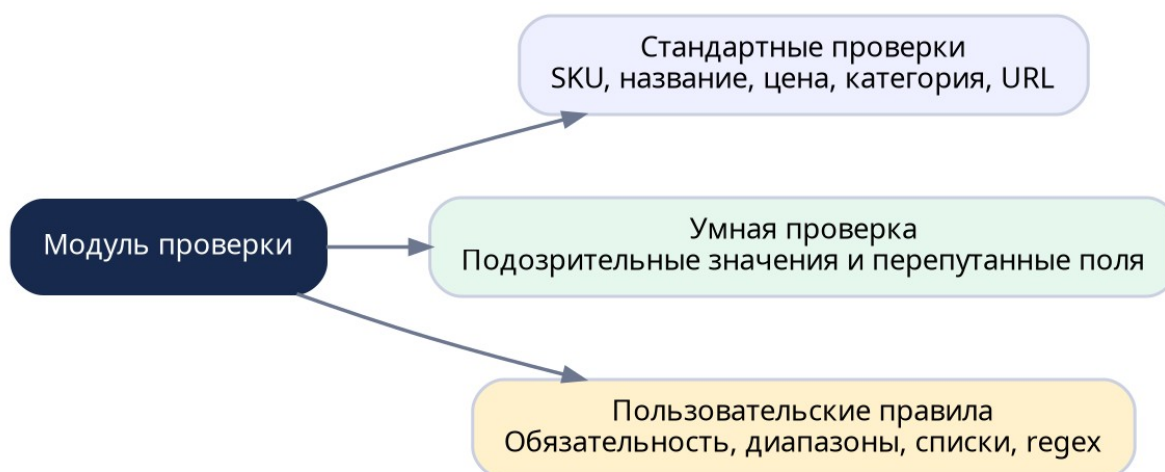
3. Импорт и рабочее состояние



Операционный поток данных в настольном приложении.

Активный каталог хранится как рабочий DataFrame. Дополнительное состояние включает путь источника, лист, исходный номер строки, сопоставление столбцов, правила, ошибки, статус товара, здоровье каталога и историю изменений. Исходные файлы используются только для чтения.

4. Архитектура проверки



Три уровня проверки используют одно рабочее состояние каталога.

Стандартная проверка находит предсказуемые структурные ошибки. Умная проверка ищет подозрительные смысловые шаблоны. Пользовательские правила позволяют задавать обязательность, диапазоны, списки и регулярные выражения без изменения кода.

5. Редактирование и история

1

Пользователь запускает ручное изменение, массовую операцию, безопасное исправление или сценарий.

2

Программа создаёт снимок для Undo.

3

Рабочий DataFrame изменяется.

4

Проверка запускается повторно.

5

Обновляются ошибки, статусы и здоровье каталога.

Принцип

Изменения должны быть обратимыми. Для очень больших каталогов в будущем можно использовать историю дельт вместо полных копий DataFrame.

6. Операции с каталогами

Сервис	Роль
Сравнение	Сопоставляет товары по SKU или другому ключу и фиксирует изменения полей.
Обновление поставщика	Обновляет совпавшие товары, добавляет новые строки и защищает заполненные значения от пустых обновлений.
Разделение	Делит один каталог по значению группы или фиксированному числу строк.
Объединение	Сводит схемы нескольких файлов и сообщает о повторяющихся ключах.
Сценарии преобразования	Выполняет упорядоченные повторяемые операции и записывает каждое изменение.

7. Пакетная архитектура



Пакетные сервисы работают как последовательный процесс и сохраняют результаты каждого файла.

Для каждого каталога сохраняются собственные данные, сопоставление, результат проверки и метаданные экспорта. Ошибка одного файла не останавливает остальные. Исправления и ручная проверка ведут подробную историю перед публикацией.

8. Публикация для маркетплейсов

- Публикация начинается с проверенного рабочего каталога.
- Режим публикации определяет включение готовых, требующих проверки или всех товаров.
- Экспортёры преобразуют данные в столбцы Shopify или WooCommerce.
- Исключённые товары можно сохранить отдельным отчётом.
- Выходные файлы по умолчанию имеют статус draft или unpublished.

9. Локальная конфиденциальность

Граница данных

Исходный каталог → локальная обработка Windows → выбранная папка результата. Для публичной архитектуры не требуется облачная загрузка или телеметрия.

10. Сборка Windows-версии



Процесс создания коммерческого Windows-продукта.

PyInstaller упаковывает приложение Python и среду выполнения в portable-папку. Inno Setup создаёт установщик. Установщик, документация и примеры помещаются в один ZIP для покупателя Gumroad.

11. Разделение репозиториев



Приватная реализация и публичная витрина намеренно разделены.

Репозиторий	Содержимое
Private-репозиторий	Полный код, тесты, скрипты сборки, конфигурация установщика и внутренние файлы релиза.
Public Showcase	README, скриншоты, публичная архитектура, Privacy, системные требования и ссылка Gumroad.

12. Возможности расширения

- Дополнительные экспортёры маркетплейсов.
- Повторяемые профили сопоставления.
- Подключаемые валидаторы.
- Проверка изображений товаров.
- API-интеграции и опциональные AI-сервисы с ключами пользователя.